



PRACTICAL - 1

Implementation and Time Analysis of Sorting Algorithms

Aim of the Practical:

To implement Bubble Sort, Selection Sort, Insertion Sort, Merge Sort and Quick Sort in C++ and analyze their time and space complexities.

1.1 Bubble Sort

Aim:

To implement optimized Bubble Sort and derive its best, average and worst-case time complexity.

Program:

```
#include <iostream>
using namespace std;
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    cout << "Original array: ";
    for (int x : a) cout << x << " ";
    bubbleSort(a, n);
    cout << "\nSorted array: ";
    for (int x : a) cout << x << " ";
}
```

Output:

Original array: 64 34 25 12 22 11 90 45 78 3
Sorted array: 3 11 12 22 25 34 45 64 78 90

Time Analysis:

Derivation of Time Complexity:

Let n be the number of elements in the array.

Outer loop runs at most $(n - 1)$ times.

For each outer-loop value i , the inner loop runs $(n - i - 1)$ times.

$$C(n) = \sum_{i=0}^{n-2} (n - i - 1)$$

Re-indexing with $k = n - i - 1$ gives:

$$C(n) = \sum_{k=1}^{n-1} k$$

$$C(n) = 1 + 2 + 3 + \dots + (n - 1)$$

$$C(n) = n(n - 1) / 2 = (n^2 - n) / 2$$

Each comparison and possible swap takes constant time, so:

$$T(n) = c_1 \cdot C(n) + c_2$$

$$T(n) = (c_1/2)n^2 - (c_1/2)n + c_2$$

Dropping constants and lower-order terms gives $T(n) = \Theta(n^2)$ for average/worst cases.



In the optimized version, if the array is already sorted, swapped remains false after the first pass. The inner loop then performs only $(n - 1)$ comparisons.

$$T_{\text{best}}(n) = c(n - 1) + c_0 = \Theta(n)$$

Final Time Analysis:

- Best Case: $\Theta(n)$ - already sorted array with the swapped flag.
- Average Case: $\Theta(n^2)$.
- Worst Case: $\Theta(n^2)$ - reverse-sorted array.
- Auxiliary Space: $\Theta(1)$.

Result:

Optimized Bubble Sort was implemented successfully. The derived complexity matches the program structure.

1.2 Selection Sort

Aim:

To implement Selection Sort and derive its time complexity from the number of comparisons.

Program:

```
#include <iostream>
using namespace std;
void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[minIndex]) minIndex = j;
        }
        swap(a[i], a[minIndex]);
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    selectionSort(a, n);
    for (int x : a) cout << x << " ";
}
```

Output:

3 11 12 22 25 34 45 64 78 90

Time Analysis:

Derivation of Time Complexity:

Let n be the number of elements.

The outer loop runs $(n - 1)$ times.

For $i = 0$, the inner loop performs $(n - 1)$ comparisons; for $i = 1$ it performs $(n - 2)$, and so on.

$$C(n) = (n - 1) + (n - 2) + \dots + 1$$

$$C(n) = \sum_{k=1}^{n-1} k$$

$$C(n) = n(n - 1) / 2$$

Unlike Bubble or Insertion Sort, the number of comparisons does not depend on the initial order.

The algorithm performs at most one swap per outer iteration:

$$S(n) = n - 1$$

Hence:

$$T(n) = c_1 \cdot n(n - 1) / 2 + c_2 \cdot (n - 1) + c_3$$

$$T(n) = \Theta(n^2)$$



Final Time Analysis:

- Best Case: $\Theta(n^2)$.
- Average Case: $\Theta(n^2)$.
- Worst Case: $\Theta(n^2)$.
- Auxiliary Space: $\Theta(1)$.

Result:

Selection Sort was implemented successfully and its quadratic time complexity was derived from the fixed number of comparisons.

1.3 Insertion Sort

Aim:

To implement Insertion Sort and derive its best and worst-case complexities.

Program:

```
#include <iostream>
using namespace std;
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    insertionSort(a, n);
    for (int x : a) cout << x << " ";
}
```

Output:

3 11 12 22 25 34 45 64 78 90

Time Analysis:

Derivation of Time Complexity:

Let n be the number of elements.

The outer loop runs from $i = 1$ to $n - 1$, so it runs $(n - 1)$ times.

Worst case: the array is in reverse order. For each i , the key must be shifted across all i previous elements.

$$C_{\text{worst}}(n) = \sum_{i=1}^{n-1} i$$

$$C_{\text{worst}}(n) = 1 + 2 + \dots + (n - 1)$$

$$C_{\text{worst}}(n) = n(n - 1)/2$$

$$T_{\text{worst}}(n) = \Theta(n^2)$$

Best case: the array is already sorted. The while-condition fails after the first comparison in every outer-loop iteration.

$$C_{\text{best}}(n) = \sum_{i=1}^{n-1} 1 = n - 1$$

$$T_{\text{best}}(n) = \Theta(n)$$

For random input, approximately half of the already-sorted portion is shifted on average, which still gives $\Theta(n^2)$.

Final Time Analysis:

- Best Case: $\Theta(n)$.
- Average Case: $\Theta(n^2)$.
- Worst Case: $\Theta(n^2)$.
- Auxiliary Space: $\Theta(1)$.

Result:

Insertion Sort was implemented successfully. The step-by-step count explains why nearly sorted data is favorable.

1.4 Merge Sort

Aim:

To implement Merge Sort using divide and conquer and solve its recurrence using the Master Method.

Program:

```
#include <iostream>
#include <vector>
using namespace std;
void merge(int a[], int l, int m, int r) {
    vector<int> temp;
    int i = l, j = m + 1;
    while (i <= m && j <= r)
        temp.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
    while (i <= m) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);
    for (int k = 0; k < (int)temp.size(); k++)
        a[l + k] = temp[k];
}
void mergeSort(int a[], int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    mergeSort(a, 0, n - 1);
    for (int x : a) cout << x << " ";
}
```

Output:

3 11 12 22 25 34 45 64 78 90

Time Analysis:

Time Complexity using Master Method:

Merge Sort divides the input into two equal halves and merges the two sorted halves in linear time.

$$T(n) = 2T(n/2) + cn$$

Compare with the standard Master Theorem form:

$$T(n) = aT(n/b) + f(n)$$

$$a = 2, \quad b = 2, \quad f(n) = cn = \Theta(n)$$

Now compute:

$$n^{(\log_b a)} = n^{(\log_2 2)} = n^1 = n$$

Since:

$$f(n) = \Theta(n) = \Theta(n^{(\log_b a)})$$

this is Master Theorem Case 2.

$$T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

$$T(n) = \Theta(n \log n)$$

The input is always divided and merged in the same manner, so best, average and worst cases are all $\Theta(n \log n)$.

Final Time Analysis:

- Best Case: $\Theta(n \log n)$.
- Average Case: $\Theta(n \log n)$.
- Worst Case: $\Theta(n \log n)$.
- Auxiliary Space: $\Theta(n)$.

Result:

Merge Sort was implemented successfully and its recurrence was solved step by step using the Master Method.

1.5 Quick Sort

Aim:

To implement Quick Sort and use the Master Method for the balanced recurrence while deriving the worst case separately.

Program:

```
#include <iostream>
using namespace std;
int partitionArray(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[high]);
    return i + 1;
}
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int p = partitionArray(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    quickSort(a, 0, n - 1);
    for (int x : a) cout << x << " ";
}
```

Output:

3 11 12 22 25 34 45 64 78 90

Time Analysis:

Time Complexity using Master Method:

Partitioning scans the current subarray once, so partition cost is $\Theta(n)$.

Best/balanced case: the pivot divides the array into two approximately equal halves.

$$T(n) = 2T(n/2) + cn$$

Using Master Theorem:

$$a = 2, \quad b = 2, \quad f(n) = \Theta(n)$$



$$n^{(\log_b a)} = n^{(\log_2 2)} = n$$

$$f(n) = \Theta(n^{(\log_b a)})$$

Hence Master Theorem Case 2 applies:

$$T_{\text{best}}(n) = \Theta(n \log n)$$

Average case also produces $\Theta(n \log n)$ expected running time.

Worst case: if the pivot is repeatedly the smallest or largest element, one side has $n - 1$ elements and the other side has 0.

$$T(n) = T(n - 1) + cn$$

This recurrence is not in $aT(n/b) + f(n)$ form, so the ordinary Master Theorem does not apply.

Unrolling:

$$\begin{aligned} T(n) &= T(n - 1) + cn \\ &= T(n - 2) + c(n - 1) + cn \\ &= T(1) + c(2 + 3 + \dots + n) \\ &= \Theta(n^2) \end{aligned}$$

Final Time Analysis:

- Best Case: $\Theta(n \log n)$.
- Average Case: $\Theta(n \log n)$.
- Worst Case: $\Theta(n^2)$.
- Average recursion stack: $\Theta(\log n)$.
- Worst-case recursion stack: $\Theta(n)$.

Result:

Quick Sort was implemented successfully. The balanced case was solved using the Master Method, and the unbalanced worst case was derived separately.

Conclusion:

All five sorting algorithms were implemented and their time complexities were derived step by step. Merge Sort and balanced/average Quick Sort achieve $\Theta(n \log n)$, while Bubble, Selection and Insertion Sort are quadratic in their general or worst cases.

PRACTICAL - 2

Implementation and Time Analysis of Linear and Binary Search

Aim of the Practical:

To implement Linear Search and Binary Search in C++ and derive their time and space complexities.

2.1 Linear Search

Program:

```
#include <iostream>
using namespace std;
int linearSearch(int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key) return i;
    return -1;
}
int main() {
    int a[] = {10, 25, 7, 40, 15, 30};
    int n = sizeof(a) / sizeof(a[0]);
    int key = 40;
    cout << linearSearch(a, n, key);
}
```

Output:

3

Time Analysis:

Derivation of Time Complexity:

Let n be the number of elements.

Best case: the key is the first element, so only one comparison is required.

$$T_{\text{best}}(n) = c = \Theta(1)$$

Worst case: the key is the last element or absent, so n comparisons are performed.

$$T_{\text{worst}}(n) = c_1 \cdot n + c_2 = \Theta(n)$$

Average case: assuming the key is equally likely to be at any position, expected comparisons are:

$$C_{\text{avg}}(n) = (1 + 2 + \dots + n) / n$$

$$C_{\text{avg}}(n) = [n(n + 1)/2] / n = (n + 1)/2$$

$$T_{\text{avg}}(n) = \Theta(n)$$

Final Time Analysis:

- Best Case: $\Theta(1)$.
- Average Case: $\Theta(n)$.
- Worst Case: $\Theta(n)$.
- Auxiliary Space: $\Theta(1)$.

Result:

Linear Search was implemented successfully and its comparison count was derived for best, average and worst cases.

2.2 Binary Search

Program:

```
#include <iostream>
using namespace std;
int binarySearch(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
```



```
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
int main() {
    int a[] = {5, 10, 15, 20, 25, 30, 35, 40};
    int n = sizeof(a) / sizeof(a[0]);
    cout << binarySearch(a, n, 30);
}
```

Output:

5

Time Analysis:

Derivation using Recurrence / Master Method:

At every step Binary Search discards half of the remaining elements.

$$T(n) = T(n/2) + c$$

Compare with $T(n) = aT(n/b) + f(n)$:

$$a = 1, \quad b = 2, \quad f(n) = \Theta(1)$$

$$n^{(\log_b a)} = n^{(\log_2 1)} = n^0 = 1$$

$$f(n) = \Theta(1) = \Theta(n^{(\log_b a)})$$

Therefore Master Theorem Case 2 applies:

$$T(n) = \Theta(\log n)$$

Equivalently, after k halvings $n/2^k = 1$, so $k = \log_2 n$.

Final Time Analysis:

- Best Case: $\Theta(1)$.
- Average Case: $\Theta(\log n)$.
- Worst Case: $\Theta(\log n)$.
- Auxiliary Space (iterative): $\Theta(1)$.

Result:

Binary Search was implemented successfully, and its logarithmic complexity follows directly from repeated halving.



PRACTICAL - 3

Implementation of Max-Heap Sort Algorithm

Aim of the Practical:

To implement Heap Sort using a max heap and analyze its running time step by step.

Algorithm:

1. Build a max heap from the input array.
2. Swap the maximum element at the root with the last element.
3. Reduce heap size by one.
4. Heapify the root.
5. Repeat until the heap becomes empty.

Program:

```
#include <iostream>
using namespace std;
void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    if (left < n && a[left] > a[largest]) largest = left;
    if (right < n && a[right] > a[largest]) largest = right;
    if (largest != i) {
        swap(a[i], a[largest]);
        heapify(a, n, largest);
    }
}
void heapSort(int a[], int n) {
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(a, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(a[0], a[i]);
        heapify(a, i, 0);
    }
}
int main() {
    int a[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(a) / sizeof(a[0]);
    heapSort(a, n);
    for (int x : a) cout << x << " ";
}
```

Output:

5 6 7 11 12 13

Time Analysis:

Derivation of Time Complexity:

Heap Sort has two main phases: build the max heap and repeatedly extract the maximum.

Phase 1 - Build Heap: heapify is called on internal nodes from $n/2 - 1$ down to 0.

The tighter analysis of build-heap sums the work by node height:

$$T_{\text{build}}(n) = \sum_{h=0}^{\log n} [\text{number of nodes of height } h] \cdot O(h)$$

$$T_{\text{build}}(n) = O(n \cdot \sum_{h \geq 0} h / 2^{(h+1)})$$

The infinite series $\sum h/2^{(h+1)}$ converges to a constant, therefore:

$$T_{\text{build}}(n) = O(n)$$

Phase 2 - Repeated extraction: there are $(n - 1)$ extractions.

Each heapify after extraction takes at most $O(\log n)$.

$$T_{\text{extract}}(n) = (n - 1) \cdot O(\log n) = O(n \log n)$$

Total:

$$T(n) = O(n) + O(n \log n) = O(n \log n)$$

Final Time Analysis:

- Best Case: $\Theta(n \log n)$.
- Average Case: $\Theta(n \log n)$.
- Worst Case: $\Theta(n \log n)$.
- Auxiliary Space: $O(\log n)$ with recursive heapify.

Result:

Max-Heap Sort was implemented successfully and its total complexity was obtained by analyzing build-heap and extraction separately.

PRACTICAL - 4

Implementation and Time Analysis of Factorial using Iterative and Recursive Methods

Aim of the Practical:

To implement factorial iteratively and recursively and compare their time and space complexities.

Program:

```
#include <iostream>
using namespace std;
long long factorialIterative(int n) {
    long long fact = 1;
    for (int i = 2; i <= n; i++)
        fact *= i;
    return fact;
}
long long factorialRecursive(int n) {
    if (n <= 1) return 1;
    return n * factorialRecursive(n - 1);
}
int main() {
    int n = 5;
    cout << "Iterative: " << factorialIterative(n) << endl;
    cout << "Recursive: " << factorialRecursive(n) << endl;
}
```

Output:

Iterative: 120
Recursive: 120

Time Analysis:

Iterative Method - Derivation:

The loop starts at 2 and runs through n.

$$\text{Number of iterations} = n - 1$$

Each iteration performs constant work.

$$T_{\text{iter}}(n) = c(n - 1) + c_0 = \Theta(n)$$

Only a fixed number of variables are used, so auxiliary space is $\Theta(1)$.

Recursive method recurrence:

$$T_{\text{rec}}(n) = T_{\text{rec}}(n - 1) + c$$

Unrolling:

$$\begin{aligned} T_{\text{rec}}(n) &= T_{\text{rec}}(n - 2) + 2c \\ &= T_{\text{rec}}(1) + (n - 1)c \\ &= \Theta(n) \end{aligned}$$

There are n recursive stack frames in the worst case, so auxiliary space is $\Theta(n)$.

Final Time Analysis:

- Iterative Time: $\Theta(n)$, Space: $\Theta(1)$.
- Recursive Time: $\Theta(n)$, Space: $\Theta(n)$.

Result:

Both methods compute factorial correctly. Their time is linear, but recursion uses linear stack space.



PRACTICAL - 5

Implementation of 0/1 Knapsack using Dynamic Programming

Aim of the Practical:

To solve the 0/1 Knapsack problem using dynamic programming and analyze the DP table construction.

Algorithm:

1. Let $dp[i][w]$ denote the best value using the first i items and capacity w .
2. Initialize row 0 and column 0 with 0.
3. If $wt[i-1] > w$, set $dp[i][w] = dp[i-1][w]$.
4. Otherwise set $dp[i][w] = \max(dp[i-1][w], value[i-1] + dp[i-1][w-wt[i-1]])$.
5. Return $dp[n][W]$.

Program:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
int knapsack(int W, const vector<int>& wt, const vector<int>& val) {
    int n = wt.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i - 1][w];
            if (wt[i - 1] <= w) {
                dp[i][w] = max(dp[i][w],
                               val[i - 1] + dp[i - 1][w - wt[i - 1]]);
            }
        }
    }
    return dp[n][W];
}

int main() {
    vector<int> wt = {1, 3, 4, 5};
    vector<int> val = {1, 4, 5, 7};
    int W = 7;
    cout << knapsack(W, wt, val);
}
```

Output:

9

Time Analysis:

Derivation of Time Complexity:

Let n be the number of items and W be the knapsack capacity.

The DP table contains $(n + 1)$ rows and $(W + 1)$ columns.

Number of states = $(n + 1)(W + 1)$

The two nested loops visit every non-base state once.

For each state, only constant-time comparisons, additions and max operations are performed.

$$T(n, W) = c \cdot n \cdot (W + 1) + c_0$$

$$T(n, W) = \Theta(nW)$$

The table stores $(n + 1)(W + 1)$ integer values.

$$S(n, W) = \Theta(nW)$$

This is pseudo-polynomial because W is a numeric capacity, not the number of bits needed to encode W .

Final Time Analysis:

- Time Complexity: $\Theta(nW)$.

-
- Space Complexity: $\Theta(nW)$.

Result:

The 0/1 Knapsack problem was solved using dynamic programming, and its complexity was derived from the number of DP states.



PRACTICAL - 6

Implementation of Matrix Chain Multiplication using Dynamic Programming

Aim of the Practical:

To find the parenthesization with minimum scalar multiplications using dynamic programming and derive its complexity.

Recurrence:

$$m[i,j] = 0, \text{ if } i = j$$

$$m[i,j] = \min\{i \leq k < j\} [m[i,k] + m[k+1,j] + p[i-1]p[k]p[j]]$$

Program:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;
long long matrixChain(const vector<int>& p) {
    int n = p.size() - 1;
    vector<vector<long long>> dp(n + 1, vector<long long>(n + 1, 0));
    for (int len = 2; len <= n; len++) {
        for (int i = 1; i <= n - len + 1; i++) {
            int j = i + len - 1;
            dp[i][j] = LLONG_MAX;
            for (int k = i; k < j; k++) {
                long long cost = dp[i][k] + dp[k + 1][j]
                    + 1LL * p[i - 1] * p[k] * p[j];
                if (cost < dp[i][j]) dp[i][j] = cost;
            }
        }
    }
    return dp[1][n];
}
int main() {
    vector<int> p = {4, 10, 3, 8};
    cout << matrixChain(p);
}
```

Output:

216

Step-by-Step Cost Calculation for the Sample:

- $A_1 = 4 \times 10$, $A_2 = 10 \times 3$, $A_3 = 3 \times 8$.
- $(A_1 A_2) A_3$: cost = $(4 \times 10 \times 3) + (4 \times 3 \times 8) = 120 + 96 = 216$.
- $A_1 (A_2 A_3)$: cost = $(10 \times 3 \times 8) + (4 \times 10 \times 8) = 240 + 320 = 560$.
- Minimum = 216 scalar multiplications.

Time Analysis:

Derivation of Time Complexity:

Let n be the number of matrices.

The outer loop chooses chain length len from 2 to n : $O(n)$ possibilities.

For a fixed len , the starting index i can take at most $O(n)$ values.

For every pair (i,j) , the split point k runs from i to $j-1$: at most $O(n)$ choices.

$$T(n) = \sum_{len} \sum_i \sum_k O(1)$$

$$T(n) = O(n \cdot n \cdot n) = O(n^3)$$

More precisely, the number of evaluated split points is:

$$\sum_{len=2 \text{ to } n} (n - len + 1)(len - 1)$$

which grows on the order of n^3 .

The DP table has $(n + 1)^2$ entries.

$$S(n) = \Theta(n^2)$$

Final Time Analysis:

- Time Complexity: $\Theta(n^3)$.
- Space Complexity: $\Theta(n^2)$.

Result:

Matrix Chain Multiplication was solved successfully. The sample minimum cost is 216 and the DP complexity is cubic in the number of matrices.

PRACTICAL - 7

Implementation of Making Change Problem using Dynamic Programming

Aim of the Practical:

To find the minimum number of coins required to make a target amount using dynamic programming and analyze its complexity.

Algorithm:

1. Initialize $dp[0] = 0$ and $dp[1...A] = \infty$.
2. For each amount x from 1 to A , try every coin denomination.
3. If $coin \leq x$ and $dp[x-coin]$ is finite, update $dp[x] = \min(dp[x], dp[x-coin] + 1)$.
4. Return $dp[A]$, or -1 if it is still infinite.

Program:

```
#include <iostream>
#include <vector>
#include <climits>
#include <algorithm>
using namespace std;
int minCoins(const vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;
    for (int x = 1; x <= amount; x++) {
        for (int coin : coins) {
            if (coin <= x && dp[x - coin] != INT_MAX) {
                dp[x] = min(dp[x], dp[x - coin] + 1);
            }
        }
    }
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}
int main() {
    vector<int> coins = {1, 3, 4};
    int amount = 6;
    cout << minCoins(coins, amount);
}
```

Output:

2

Step-by-Step DP for coins {1,3,4}, amount 6:

- $dp[0] = 0$
- $dp[1] = 1$ using {1}
- $dp[2] = 2$ using {1,1}
- $dp[3] = 1$ using {3}
- $dp[4] = 1$ using {4}
- $dp[5] = 2$ using {4,1}
- $dp[6] = 2$ using {3,3}

Time Analysis:

Derivation of Time Complexity:

Let A be the target amount and m be the number of coin denominations.

The outer loop runs once for each amount from 1 to A .

Outer-loop iterations = A

For every amount, the algorithm checks all m coin denominations.

Inner-loop iterations per amount = m



Each inner-loop operation performs a constant-time comparison and possible update.

$$T(A,m) = A \cdot m \cdot O(1)$$

$$T(A,m) = \Theta(Am)$$

The dp array contains $A + 1$ entries.

$$S(A) = \Theta(A)$$

Final Time Analysis:

- Time Complexity: $\Theta(Am)$.
- Space Complexity: $\Theta(A)$.

Result:

The Making Change problem was solved successfully using dynamic programming, and the minimum number of coins for amount 6 is 2.



PRACTICAL - 8

Implementation of Graph Searching using DFS and BFS

Aim of the Practical:

To implement Depth First Search (DFS) and Breadth First Search (BFS) for graph traversal and analyze their time and space complexities.

Algorithm:

1. Represent the graph using an adjacency list.
2. For BFS, use a queue and visit vertices level by level.
3. For DFS, use recursion and visit an unvisited adjacent vertex before backtracking.
4. Maintain a visited array so that every vertex is processed only once.

Program:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;
void bfs(int start, const vector<vector<int>>& adj) {
    vector<bool> visited(adj.size(), false);
    queue<int> q;
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        cout << u << " ";
        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}

void dfs(int u, const vector<vector<int>>& adj, vector<bool>& visited) {
    visited[u] = true;
    cout << u << " ";
    for (int v : adj[u])
        if (!visited[v]) dfs(v, adj, visited);
}

int main() {
    vector<vector<int>> adj = {
        {1,2}, {0,3,4}, {0,4}, {1}, {1,2}
    };
    cout << "BFS: ";
    bfs(0, adj);
    vector<bool> visited(adj.size(), false);
    cout << "\nDFS: ";
    dfs(0, adj, visited);
}
```

Output:

BFS: 0 1 2 3 4
DFS: 0 1 3 4 2

Time Analysis:

Step-by-Step Derivation:

Let V be the number of vertices and E be the number of edges.

With an adjacency list, initializing the visited array requires $\Theta(V)$.

Each vertex is inserted/removed from the BFS queue at most once, or entered by DFS at most once.

Vertex-processing cost = $\Theta(V)$



Across all adjacency lists, every undirected edge appears twice (once at each endpoint), so all neighbor scans together require $\Theta(E)$.

Edge-scanning cost = $\Theta(E)$

$$T(V, E) = \Theta(V) + \Theta(E) = \Theta(V + E)$$

BFS stores the queue and visited array; DFS stores the visited array and recursion stack. Both can require $\Theta(V)$ auxiliary space.

Final Time Analysis:

- BFS Time: $\Theta(V + E)$.
- DFS Time: $\Theta(V + E)$.
- BFS Space: $\Theta(V)$.
- DFS Space: $\Theta(V)$.

Result:

BFS and DFS were implemented successfully, and their linear graph-traversal complexity was derived from vertex and edge processing.



PRACTICAL - 9

Implementation of Prim's Algorithm

Aim of the Practical:

To implement Prim's algorithm to find a Minimum Spanning Tree (MST) of a connected weighted undirected graph and analyze its complexity.

Program:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;
int main() {
    const int V = 5;
    int g[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    };
    vector<int> key(V, INT_MAX), parent(V, -1);
    vector<bool> used(V, false);
    key[0] = 0;
    for (int count = 0; count < V - 1; count++) {
        int u = -1;
        for (int v = 0; v < V; v++)
            if (!used[v] && (u == -1 || key[v] < key[u])) u = v;
        used[u] = true;
        for (int v = 0; v < V; v++)
            if (g[u][v] && !used[v] && g[u][v] < key[v]) {
                parent[v] = u;
                key[v] = g[u][v];
            }
    }
    int cost = 0;
    for (int v = 1; v < V; v++) {
        cout << parent[v] << " - " << v << " : " << g[v][parent[v]] << endl;
        cost += g[v][parent[v]];
    }
    cout << "MST cost = " << cost;
}
```

Output:

```
0 - 1 : 2
1 - 2 : 3
0 - 3 : 6
1 - 4 : 5
MST cost = 16
```

Time Analysis:

Step-by-Step Derivation:

Let V be the number of vertices. This implementation uses an adjacency matrix and a linear scan to select the minimum-key unvisited vertex.

The outer loop executes $V - 1$ times.

During each iteration, selecting the next minimum-key vertex scans all V vertices.

$$\text{Selection cost} = (V - 1) \cdot \Theta(V) = \Theta(V^2)$$

After selecting u , the algorithm scans row u of the adjacency matrix, again checking V possible neighbors.

$$\text{Relaxation cost} = (V - 1) \cdot \Theta(V) = \Theta(V^2)$$

$$T(V) = \Theta(V^2) + \Theta(V^2) = \Theta(V^2)$$

The matrix itself requires $\Theta(V^2)$ storage; the additional key, parent and used arrays require $\Theta(V)$.

Final Time Analysis:

- Time Complexity (matrix implementation): $\Theta(V^2)$.
- Auxiliary Space excluding graph: $\Theta(V)$.
- Graph storage using matrix: $\Theta(V^2)$.

Result:

Prim's algorithm produced an MST of total cost 16 for the sample graph.



PRACTICAL - 10

Implementation of Kruskal's Algorithm

Aim of the Practical:

To implement Kruskal's algorithm using Disjoint Set Union (DSU) and analyze the cost of sorting and cycle detection.

Program:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
struct Edge { int u, v, w; };
struct DSU {
    vector<int> p, rankv;
    DSU(int n): p(n), rankv(n,0) { for(int i=0;i<n;i++) p[i]=i; }
    int find(int x) { return p[x]==x ? x : p[x]=find(p[x]); }
    bool unite(int a,int b) {
        a=find(a); b=find(b);
        if(a==b) return false;
        if(rankv[a]<rankv[b]) swap(a,b);
        p[b]=a;
        if(rankv[a]==rankv[b]) rankv[a]++;
        return true;
    }
};
int main() {
    int V = 4;
    vector<Edge> edges = {
        {0,1,10},{0,2,6},{0,3,5},{1,3,15},{2,3,4}
    };
    sort(edges.begin(), edges.end(),
        [](Edge a, Edge b){ return a.w < b.w; });
    DSU dsu(V);
    int cost = 0;
    for (auto e : edges) {
        if (dsu.unite(e.u, e.v)) {
            cout << e.u << " - " << e.v << " : " << e.w << endl;
            cost += e.w;
        }
    }
    cout << "MST cost = " << cost;
}
```

Output:

```
2 - 3 : 4
0 - 3 : 5
0 - 1 : 10
MST cost = 19
```

Time Analysis:

Step-by-Step Derivation:

Let V be the number of vertices and E the number of edges.

First, all E edges are sorted by weight.

$$\text{Sorting cost} = \Theta(E \log E)$$

Then each edge is considered at most once. For each edge, DSU performs find/unite operations.

With path compression and union by rank, the amortized cost of a DSU operation is $O(\alpha(V))$, where α is the inverse Ackermann function.

$$\text{DSU processing cost} = O(E \alpha(V))$$

$$T(V,E) = \Theta(E \log E) + O(E \alpha(V))$$

Since sorting dominates:



$$T(V,E) = \Theta(E \log E)$$

For a simple graph $E \leq V^2$, $\log E = O(\log V)$, so this is also commonly written $O(E \log V)$.

Final Time Analysis:

- Time Complexity: $\Theta(E \log E)$, dominated by sorting.
- DSU auxiliary storage: $\Theta(V)$.
- Edge-list storage: $\Theta(E)$.

Result:

Kruskal's algorithm successfully constructed the MST and avoided cycles using DSU.



PRACTICAL - 11

Implementation of Floyd-Warshall Algorithm

Aim of the Practical:

To implement the Floyd-Warshall algorithm for all-pairs shortest paths and derive its cubic time complexity.

Program:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;
int main() {
    const int INF = 1000000;
    vector<vector<int>>> d = {
        {0, 5, INF, 10},
        {INF, 0, 3, INF},
        {INF, INF, 0, 1},
        {INF, INF, INF, 0}
    };
    int V = d.size();
    for (int k = 0; k < V; k++)
        for (int i = 0; i < V; i++)
            for (int j = 0; j < V; j++)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
    for (auto &row : d) {
        for (int x : row)
            cout << (x >= INF ? -1 : x) << " ";
        cout << endl;
    }
}
```

Output:

```
0      5      8      9
-1     0      3      4
-1    -1     0      1
-1    -1    -1     0
```

Time Analysis:

Step-by-Step Derivation:

Let V be the number of vertices.

The algorithm has three nested loops over k, i and j.

k-loop iterations = V

i-loop iterations for each k = V

j-loop iterations for each (k,i) = V

Inside the innermost loop, only constant-time arithmetic, comparison and assignment are performed.

$$T(V) = V \cdot V \cdot V \cdot \Theta(1)$$

$$T(V) = \Theta(V^3)$$

The distance table contains V^2 entries.

$$S(V) = \Theta(V^2)$$

Final Time Analysis:

- Time Complexity: $\Theta(V^3)$.
- Space Complexity: $\Theta(V^2)$.

Result:

Floyd-Warshall successfully computed the shortest distance between every ordered pair of vertices.



Marwadi
University
Marwadi Chandarana Group



MARWADI UNIVERSITY
DEPARTMENT OF AI, ML
DAA (01AI0506)
Lab Manual



PRACTICAL - 12

Implementation of Travelling Salesman Problem

Aim of the Practical:

To solve the Travelling Salesman Problem (TSP) using dynamic programming with bitmasking and analyze its exponential state space.

Concept:

The DP state $dp[mask][u]$ stores the minimum cost to start at vertex 0, visit exactly the cities represented by mask, and finish at city u.

Program:

```
#include <iostream>
#include <vector>
#include <climits>
using namespace std;
int main() {
    vector<vector<int>> cost = {
        {0,10,15,20},
        {10,0,35,25},
        {15,35,0,30},
        {20,25,30,0}
    };
    int n = cost.size();
    int N = 1 << n;
    const int INF = 1000000000;
    vector<vector<int>> dp(N, vector<int>(n, INF));
    dp[1][0] = 0;
    for (int mask = 1; mask < N; mask++) {
        for (int u = 0; u < n; u++) {
            if (!(mask & (1 << u)) || dp[mask][u] == INF) continue;
            for (int v = 0; v < n; v++) {
                if (mask & (1 << v)) continue;
                int nextMask = mask | (1 << v);
                dp[nextMask][v] =
                    min(dp[nextMask][v], dp[mask][u] + cost[u][v]);
            }
        }
    }
    int ans = INF;
    int full = N - 1;
    for (int u = 1; u < n; u++)
        ans = min(ans, dp[full][u] + cost[u][0]);
    cout << "Minimum tour cost = " << ans;
}
```

Output:

Minimum tour cost = 80

Time Analysis:

Step-by-Step Derivation:

Let n be the number of cities.

A bitmask has 2^n possible subsets of cities.

For each subset mask, the DP stores a value for each possible current city u.

Number of DP states = $n \cdot 2^n$

For each state, the transition may try up to n unvisited next cities.

Work per state = $O(n)$

$T(n) = O(n \cdot 2^n \cdot n)$

$T(n) = O(n^2 \cdot 2^n)$

The DP table contains $n \cdot 2^n$ entries.

$$S(n) = O(n \cdot 2^n)$$

This is substantially better than checking all $n!$ tours by brute force, but it is still exponential.

Final Time Analysis:

- Time Complexity: $O(n^2 \cdot 2^n)$.
- Space Complexity: $O(n \cdot 2^n)$.

Result:

The dynamic-programming TSP implementation found a minimum tour cost of 80 for the sample graph.